

Artificial Intelligence_CSA1718

Lab Experiments 7 & 8

Experiment 7

Title

Write a Python Program to Implement Depth First Search (DFS)

Aim

To implement the **Depth First Search (DFS)** algorithm using Python to traverse all the vertices of a graph.

Objective

- To understand the working of the DFS algorithm.
- To implement graph traversal using recursion.
- To visit every node in a graph exactly once.

Theory

Depth First Search (DFS) is a graph traversal algorithm that explores a graph by visiting one branch completely before backtracking. It uses a **stack** data structure, either explicitly or through recursion. DFS is widely used in Artificial Intelligence, graph theory, maze solving, path finding, cycle detection, and topological sorting.

Applications

- Maze solving
- Path finding
- Cycle detection
- Topological sorting
- Artificial Intelligence
- Network traversal

Algorithm

1. Start from the source vertex.
2. Mark the source vertex as visited.
3. Print the current vertex.
4. Visit all adjacent unvisited vertices recursively.
5. Continue until all vertices are visited.
6. Stop.

Pseudo Code

DFS (Graph, Vertex)

Mark Vertex as visited

Print Vertex

For each adjacent vertex

 If vertex is not visited

 DFS (adjacent vertex)

End

Python Code

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],
```

```

'D': [],
'E': ['F'],
'F': []
}

visited = set()

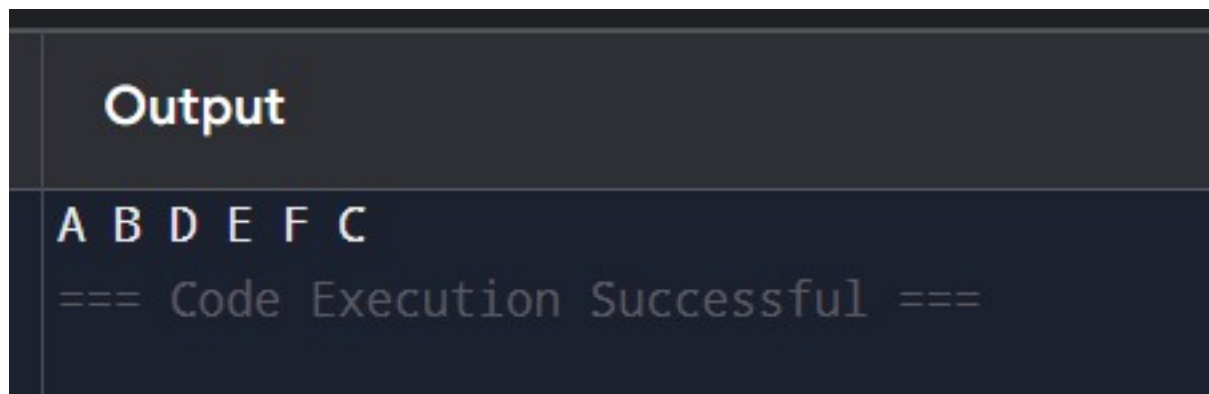
def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(neighbour)

print("Depth First Search Traversal:")
dfs('A')

```

Screenshot of the Output



Result

Thus, the Depth First Search (DFS) algorithm was successfully implemented in Python, and all vertices were traversed using recursive graph traversal.

Conclusion

DFS efficiently explores every possible path before backtracking. It is widely used in Artificial Intelligence and graph-based applications because of its simplicity and recursive implementation.

EXPERIMENT – 8

Title

Write a Python Program to Implement the Travelling Salesman Problem (TSP)

Aim

To implement the Travelling Salesman Problem using Python and determine the minimum travel cost.

Objective

- To understand the Travelling Salesman Problem.
- To find the shortest route that visits every city exactly once.
- To calculate the minimum travel cost.

Theory

The Travelling Salesman Problem (TSP) is one of the most famous optimization problems in Artificial Intelligence. The objective is to determine the shortest possible route that visits each city exactly once and returns to the starting city.

TSP is an NP-Hard problem and is widely used in logistics, delivery services, robotics, and route optimization.

Applications

- Delivery routing
- GPS navigation
- Logistics
- Robotics
- Airline scheduling

Algorithm

1. Define the cost matrix.
2. Select the starting city.
3. Generate all possible routes.
4. Calculate the total cost of each route.
5. Compare all route costs.
6. Select the route at the minimum cost.
7. Display the optimal path and total cost.

Pseudo Code

Read Cost Matrix

Generate all possible routes

For each route

 Calculate Total Cost

Store Minimum Cost

Display Best Route

Stop

Flowchart

Start

|



Input Cost Matrix



Generate Possible Routes



Calculate Total Cost



Compare Route Costs



Select Minimum Cost



Display Best Route



Stop

Python Code

```
from itertools import permutations
```

```
cities = ['A', 'B', 'C', 'D']
```

```
graph = {  
    ('A', 'B'): 10,  
    ('A', 'C'): 15,  
    ('A', 'D'): 20,  
    ('B', 'A'): 10,  
    ('B', 'C'): 35,  
    ('B', 'D'): 25,  
    ('C', 'A'): 15,  
    ('C', 'B'): 35,  
    ('C', 'D'): 30,  
    ('D', 'A'): 20,  
    ('D', 'B'): 25,  
    ('D', 'C'): 30  
}
```

```
start = 'A'
```

```
min_cost = float('inf')
```

```
best_route = None
```

```
for route in permutations([c for c in cities if c != start]):
```

```
    current_cost = 0
```

```
    current_city = start
```

```
    for city in route:
```

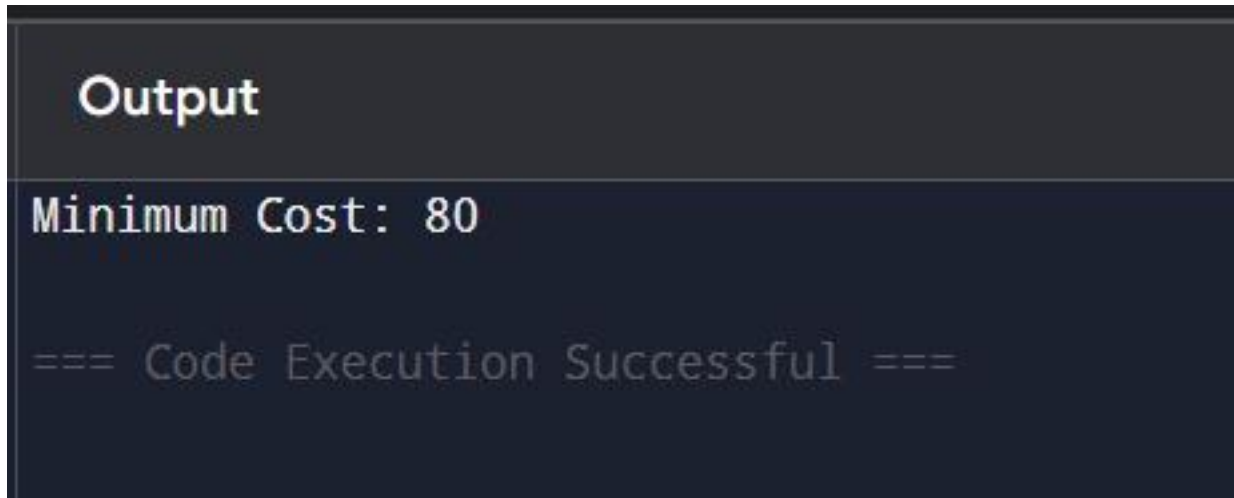
```
current_cost += graph[(current_city, city)]
current_city = city

current_cost += graph[(current_city, start)]

if current_cost < min_cost:
    min_cost = current_cost
    best_route = (start,) + route + (start,)

print("Best Route:")
print("-> ".join(best_route))
print("Minimum Cost:", min_cost)
```

Output Screenshot:

A screenshot of a code execution environment. At the top, the word "Output" is displayed in a large, white, sans-serif font. Below it, the text "Minimum Cost: 80" is shown in a smaller, white, monospace font. At the bottom, the text "=== Code Execution Successful ===" is displayed in a light gray, monospace font. The background is dark gray.

Result

Thus, the Travelling Salesman Problem was successfully implemented using Python, and the optimal route with the minimum travel cost was obtained.

Conclusion

The Travelling Salesman Problem demonstrates how Artificial Intelligence can solve route optimization problems. It is widely used in logistics, transportation, and delivery systems to reduce travel cost and improve efficiency.